Project Sun Phase 2 — Demo Q&A

Questions asked while preparing for the demo, with the answers exactly as given (11 Sep 2026). The detailed reference is `docs/Project_Sun_Phase2_Demo_Reference.pdf`.

1 · How does it crawl? What's the mechanism? 1
2 · What is high / low confidence, needs review? 1
3 · What is the scoring mechanism? 2
4 · What is structured data? 2
5 · For Nike, did you use the HTTP crawler or Playwright? 2
6 · How do you select topics? 2
7 · While selecting topics, what do you look for? 3
8 · How do you find competitors? 3
9 · How do you filter based on location? What's in the dropdown and why? 3
10 · Does language actually change anything? 3
11 · How have you designed Prompt Intelligence? 4
12 · Are Gemini calls made with API keys? Is it free? How many calls are allowed? 4
13 · Which Gemini model is used? 5
14 · Measure: how, which models, why API not browsing, cost, estimate for 1,000 / 10,000 prompts 5
15 · Legal grounds? A feasible client price, and competitor pricing 6

1 · How does it crawl? What's the mechanism?

The crawler is a bounded, planned crawl of the submitted site's public pages using plain HTTP requests, not a browser. It lives in `src/lib/crawler/orchestrator.ts`.

Sequence:

1. **Start (`POST /api/projects/:id/crawl`).** The API returns 202 at once and the crawl runs in the background. Progress is written to the `events` table and streamed to the page.
2. **Check the URL.** `https://` is added if missing, and the registrable domain is extracted with `tldts`. The hostname is resolved via DNS, and IP literals, private/loopback/link-local and cloud-metadata addresses are rejected. This check runs again before every fetch and every redirect hop.
3. **robots.txt.** It is fetched and parsed per RFC 9309 (wildcards, longest match wins) as user agent `ProjectSunBot/1.0`. If crawling is disallowed, the crawl stops as "blocked" and the profile can be filled in by hand. Any crawl-delay is honoured, up to a cap.
4. **Sitemap.** URLs come from the sitemaps robots.txt declares, otherwise from the usual paths (`/sitemap.xml` etc.). Sitemap indexes are followed, but children are sampled evenly so one product sitemap can't dominate.
5. **Planning.** Every discovered URL gets a score:
   - page type guessed from the URL and link text (home, about, product listing, pricing, locations, FAQ…);
   - whether it's in the sitemap, primary navigation or footer, and how many pages link to it;
   - depth.

   Diversity quotas per page type and per URL family (e.g. `/products/<sku>`, at most 6 pages, at most 30% of the budget) stop a shop's thousands of product pages eating the budget. It only follows links on the same registrable domain, and skips traps such as login, cart and search pages.
6. **Fetch.** Pages are fetched in parallel batches of 4 with `fetch`, following redirects manually so each hop is re-checked. Limits: 12 s timeout, 2 MB, HTML only.
7. **JavaScript fallback.** If static HTML is basically an empty JS shell, Playwright headless Chromium renders it. This happens for at most 3 pages and is skipped quietly if Chromium isn't installed.
8. **Extract.** Cheerio pulls title, meta and Open Graph tags, canonical, hreflang, headings, navigation, breadcrumbs, JSON-LD / microdata / RDFa and main text, stripping boilerplate. Exact and near-duplicate pages (similarity ≥0.88) add no evidence.
9. **Evidence.** Each page becomes up to 90 typed evidence items (source type × page type × field hint, e.g. "JSON-LD Organization on home → brand name"), plus 1,500-character text chunks. The raw text is also saved as a content-addressed file.
10. **Profile fills in as it goes.** After the first page, and every 2 pages after that, the deterministic resolvers rebuild a partial profile. That is why the Understand screen fills in live.

Default limits: 400 URLs in the queue, 30 fetched, 26 used as evidence, depth 2, 90 s total. They can be changed through `SUN_CRAWL_*` environment variables in `src/lib/crawler/config.ts`.

Observed: 9–38 s per site, e.g. nike.com used 17 pages in 20 s and hubspot.com 26 pages in about 28 s.

2 · What is high / low confidence, needs review?

Those are the badges next to each field on the Understand page, based on a 0–100 confidence score:

- **High confidence:** 75 or more
- **Medium confidence:** 50–74
- **Low confidence:** 1–49
- **Needs review:** 0, meaning no evidence was found for the field
- **Your edit:** you changed the field yourself, so it's set to 100

The score comes from the website evidence, not from the AI saying how sure it is. It's higher when:

- the fact comes from a strong source (structured data beats plain text);
- several pages agree;
- structured data and visible text say the same thing.

Conflicting evidence lowers it.

3 · What is the scoring mechanism?

The confidence score for a profile field (`src/lib/profile/confidence.ts`) is made of four parts:

1. **Source strength, up to 70 points:** 70 × the strength of the best piece of evidence. Structured data counts most (JSON-LD 1.0, microdata 0.85, hreflang 0.85, addresses 0.8, breadcrumbs 0.75, navigation 0.72, meta tags 0.6); plain page text counts less.
2. **Agreement across pages, up to 18 points:** 7 for each extra page that says the same thing.
3. **Different kinds of evidence, up to 8 points:** 3 for each extra source type.
4. **Structured data and visible text agree:** 6 points.

It then subtracts 18 for each conflicting piece of evidence, and the result is capped between 0 and 100.

The AI's own confidence is never used.

4 · What is structured data?

Structured data is information a website puts in its code specifically for machines, like Google, to read. Visitors don't see it on the page. It uses the Schema.org vocabulary, for example:

```
{ "@type": "Organization", "name": "Nike", "url": "https://nike.com", "areaServed": "India" }
```

It comes in three formats, and the crawler reads all of them (`src/lib/crawler/structured-data.ts`):

- **JSON-LD:** a script block, the most common format.
- **Microdata:** attributes such as `itemprop` added to HTML tags.
- **RDFa:** attributes such as `property` added to HTML tags.

It gets the highest weight because the business itself states the facts explicitly: brand name, products, addresses, what kind of business it is. The app doesn't have to guess them from page text.

5 · For Nike, did you use the HTTP crawler or Playwright?

Both, but almost all Nike pages came through the plain HTTP crawler. Playwright was only a fallback for a single page.

Site	HTTP pages	Playwright pages
nike.com (6 runs)	18 each	1 each
nike.in (8 runs)	29–30 each	0 or 1

- Every Nike run except one tried the Playwright fallback on some page, because that page's HTML came back as a near-empty JavaScript shell.
- The rendered version is only used if it has more content than the plain HTML. Where the table shows 0 Playwright pages, the rendered copy wasn't better and wasn't used.
- One nike.in run (06:31) had no fallback and only 5 pages; that crawl was interrupted.

6 · How do you select topics?

Topics are built without AI, from the confirmed brand profile (`src/lib/landscape/topics.ts`):

1. **Start from the profile:** its Products & services list and the customer questions found on the site.
2. **Group similar names:** similar items are clustered together. A shared end word is ignored when comparing, so "CRM Software" and "Sales Software" stay separate topics.
3. **Name each group after a product:** a customer question can join a group but can never name one, which stops FAQ questions becoming topics.
4. **Rank:** groups with the most items come first, and the app keeps up to 6.
5. **Top up if needed:** if fewer than 4 groups exist, standard topics for the business type are added (e.g. "Pricing & availability"). These are marked as having no on-site evidence.

You then confirm, remove or add topics, and need 4–6 confirmed to continue. A topic you add is confirmed straight away.

7 · While selecting topics, what do you look for?

A good topic is:

1. **Something the business actually sells.** It must come from the site's own products or services, not be invented.
2. **A category, not a single item.** "Running Shoes" rather than one shoe model or a price filter.
3. **A noun, not a question.** FAQ lines like "How do I track my order?" are dropped.
4. **Backed by more than one mention.** Categories that appear across more items and pages rank higher.
5. **Distinct from the other topics.** Near-duplicates are merged, but "Sales Software" and "CRM Software" stay separate.
6. **Free of the brand name.** Each topic also gets a brand-neutral version (e.g. "Nike Running" becomes "Running") so it can be used in Discovery prompts.

The code doesn't reject everything a person would. Product lines like "Jordan" or labels like "Gift Cards" can still get through, which is why you confirm or remove topics yourself.

8 · How do you find competitors?

Four sources run at the same time, then every candidate is checked and scored (`src/lib/landscape/`):

1. **The brand's own website:** sentences like "X vs Y", "alternative to Y" or "switching from Y".
2. **Gemini:** asked for likely rivals based on the confirmed profile. Its answers are treated as guesses and capped at 80 confidence.
3. **Wikipedia and Wikidata (free, no key):**
   - finds the brand from its official website;
   - drops its own sub-brands;
   - ranks companies that share the brand's specific Wikipedia categories, e.g. "Athletic shoe brands";
   - also uses how well known each company is and how similar its article text is.
4. **Web search:** built but switched off.

Then:

- **Merge** duplicates by name or domain.
- **Verify:** fetch the rival's homepage plus up to 3 key pages to confirm it's the right company.
- **Score:** shared market categories (highest weight), then overlap in products, customer needs, category, audience and geography.
- **Classify** each as Direct, Substitute, Adjacent or Marketplace, and drop suppliers, partners, the brand itself and weak matches.
- **Show** the top 16 as suggestions.

You confirm, reject or add rivals. Marketplaces are never compared.

9 · How do you filter based on location? What's in the dropdown and why?

There's no location filter in the app. Location is used in three places.

1. The "Market" dropdown on the start page: Global, AE (UAE), US, UK, IN (India), SA (Saudi Arabia), EU.

- It's a fixed list in `src/app/trial/new/page.tsx` . The code and docs don't record why these seven were picked.
- **It's only saved, never used.** No later step reads it: not the crawl, profile, competitors, prompts or Measure.

2. The "Target market" levels on the Understand page: city, region, country, multi-country, global.

- The markets themselves come from the website: country domain, `/in/` -style paths, language versions, store addresses, currency, phone codes.
- A head-office address, a `.com` domain or the site's language is never taken as the market.

3. Where location actually changes results:

- **Competitors:** for businesses customers visit (restaurants, clinics, local services, dealerships), geography carries more weight. A rival that doesn't serve the same area is rejected.
- **Prompts:**
  - A place only appears in a prompt if it's a confirmed location.
  - Cities are only used for businesses people visit.
  - "Near me"-type prompts are rejected.
  - Budgets use the market's currency.
- **Measure:** no location breakdown, and nothing records where an answer was asked from.

10 · Does language actually change anything?

Mostly no. It's recorded, but the output barely changes.

Step	What language does
Crawl	Only logged when the crawl starts. It doesn't pick language versions of pages.
Competitors	Sent to Gemini as a hint alongside geography. A weak effect at most.
Prompts	Saved as a label on each prompt. The instructions to Gemini and the templates are in English, so prompts are still written in English.
Collection	Saved as a label on each answer. ChatGPT and Gemini aren't told which language to use.
Analysis	English only: negation, sentiment and recommendation rules are English word lists.

So choosing Arabic, Hindi, French or Spanish gives you English prompts, analysed with English rules.

11 · How have you designed Prompt Intelligence?

Prompt Intelligence works in five stages. AI writes the wording; code decides what to ask and what to keep.

1. Freeze the inputs
The confirmed profile, topics and competitors are saved as a snapshot, so changing them later can't quietly alter the prompts.

2. Plan what to measure (code)
The set is fixed at 10 prompts per section:

- **Discovery:** no brand named. Does the AI recommend you unprompted?
- **Trade Offs:** you against one confirmed rival.
- **Sentiment Analysis:** your brand by name, on a concern like returns or authenticity.

The business type (retail, restaurant, SaaS…) sets what buyers choose between, the decision factors and the themes.

3. Build a customer picture (Gemini, 1 call)

- 6–9 real shopping needs, each with situations, qualities, audiences and a budget.
- A need is only kept if it matches words in the site's own navigation, so nothing is invented.
- Plus buyer concerns and decision factors.

4. Write candidates (Gemini, 3 calls, plus templates)

- **Discovery:** code picks one detail per prompt (a situation, audience, budget or nothing) and Gemini only writes the sentence.
- **Trade Offs:** Gemini first says which rival competes on which need; pairings outside that are discarded.
- **Sentiment:** one prompt per concern.
- Templates always produce a backup pool, and are the only source when there's no key.

This gives about 275–320 candidates.

5. Filter and select (code)

- **Rules:** no brand or rival names in Discovery, a rival and a decision factor in Trade Offs, no "near me", no leading or time-sensitive wording, no personal medical or financial advice.
- **Natural-language check:** must read like something a person would type.
- **Remove duplicates:** exact and near-identical.
- **Score out of 100:** how natural and coherent it is, how relevant, how clear, how useful to measure.
- **Pick 10 per section:** balance quality, new coverage and variety, with caps per rival and sentence style. AI-written prompts are picked first; templates only fill gaps.

Your review

You can open **Why?**, or **Edit**, **Regenerate**, **Replace**, **Deselect**, **Reject** or **Select**. Every change is re-checked against the same rules. **Approve** re-validates all 30 and locks the set.

Why it's built this way: AI output is only a draft. The rules, scoring and selection are plain code, so the prompt set can be defended and reproduced.

12 · Are Gemini calls made with API keys? Is it free? How many calls are allowed?

Yes, all Gemini calls use an API key.

Key: `GEMINI_API_KEY` from Google AI Studio, set in `.env.local` or pasted in the app. Calls go through Google's `@google/genai` library to the model `gemini-flash-lite-latest`.

Cost: free tier, no card. Two catches:

- Google may use free-tier data to improve its products, so the app only sends public website text and generated buyer questions.
- Web-search answers (grounding) aren't available on the free tier.

Calls per project:

Step	Gemini calls
Brand profile	1
Competitors	1
Prompt Intelligence	4: 1 customer picture + 3 sections (plus a retry if a section returns nothing)
Each **Regenerate** click	1
Collecting Gemini answers	~30–33
Total	**≈ 36–40**

Limits: Google sets per-minute and per-day limits for each model on the free tier. The repo doesn't record the exact numbers, so check your limits in AI Studio. The daily limit resets at midnight Pacific (12:30 IST).

- The app sends at most 14 requests a minute and 3 at once when collecting answers.
- If Google says the quota is used up, collection pauses and resumes later.
- Flash-Lite was chosen because the larger Flash model's free daily quota ran out after a single 30-prompt run.

Without a key, the app still works: profile and prompts fall back to rules and templates, and there's no Gemini lane for answers.

13 · Which Gemini model is used?

Every Gemini call uses `gemini-flash-lite-latest`: profile, competitors, prompts and collected answers alike. It's set in `src/lib/gemini/client.ts`.

- It's a floating alias: Google points it at the current Flash-Lite version.
- A pinned version was tried first, but `gemini-2.0-flash` was retired and `gemini-2.5-flash` refused new keys.
- To keep runs traceable, the model name returned with each collected answer is saved alongside it.
- In your stored answers, Gemini replies are labelled `gemini-flash-lite-latest`, plus a few `gemini-flash-latest` from an earlier setting.

14 · Measure: how, which models, why API not browsing, cost, estimate for 1,000 / 10,000 prompts

How Measure works

1. Each of the 30 approved prompts is sent once to ChatGPT and once to Gemini: 60 answers.
2. Each answer is stored, then analysed by rule-based code (no AI): brands mentioned, recommended or not, who wins comparisons, sentiment, cited sites.
3. Metrics are computed from those results: recommended unprompted, share of voice, head-to-head verdicts, reputation, sources. Missing answers are left out, never counted as 0.

Models used to collect answers

Engine	Automatic (API)	By hand
ChatGPT	OpenAI `gpt-5-nano` (paid). Free fallbacks: `gpt-oss-120b` on Cerebras or Groq	chatgpt.com, pasted back
Gemini	`gemini-flash-lite-latest` (free tier)	gemini.google.com, pasted back

Why API and not browser automation

- OpenAI's consumer terms ban automatically extracting output, Google's terms restrict automated access, and both sites block bots from their chat pages in robots.txt.
- Browser bots also break on logins, CAPTCHAs and page changes.
- APIs are the approved route, and are fast and reliable (60 answers in about 2 minutes).
- The trade-off: API answers aren't identical to what the apps show, so they're labelled as automatic, and app answers can be added by hand.

Cost per answer (current list prices)

- **gpt-5-nano:** $0.05 per 1M input tokens, $0.40 per 1M output tokens. About 150 input and 1,200–2,200 output tokens (including hidden reasoning), so ≈ **$0.0005–0.0009**.
- **Gemini Flash-Lite:** free today. On the paid tier, about 650–1,000 output tokens:
  - 3.5 Flash-Lite ($0.30 in / $2.50 out) ≈ **$0.0017–0.0025**
  - 3.1 Flash-Lite ($0.25 in / $1.50 out) ≈ **$0.001–0.0015**
  - The `-latest` alias moves between versions, so paid cost is somewhere in that range.

Estimate: 1 prompt = 1 answer per engine

Prompts	ChatGPT (nano)	Gemini 3.5 FL	Both engines
1,000	$0.50–0.90	$1.70–2.50	**≈ $2.20–3.40**
10,000	$5–9	$17–25	**≈ $22–34**

With web search, so answers include sources (not enabled today)

- **OpenAI:** +$10 per 1,000 searches, plus the searched page content billed as input tokens. About $11 per 1,000 answers, $110 per 10,000.
- **Gemini grounding:** 5,000 free searches a month, then $14 per 1,000. About $16 per 1,000 answers; about $90 per 10,000 (first 5,000 searches free).

Token counts are estimates from stored answers (average ChatGPT answer ≈ 2,000 characters, Gemini ≈ 2,600). Analysis and metrics cost nothing extra.

15 · Legal grounds? A feasible client price, and competitor pricing

Legal: yes, considered from the start (§11 of the reference)

- **Consumer apps:** OpenAI's terms ban automated extraction of output, Google restricts automated access, and both block their chat pages in robots.txt. So no browser automation; apps are only asked by hand.
- **APIs:** the approved route, and answers are labelled as API answers, not app answers.
- **Gemini with Google Search grounding:** Google's API terms restrict analysing grounded results. That's a **legal blocker** for a paid Gemini-with-sources lane, not just a cost question.
- **Production:** get written collection rights or licensed partners, plus a legal review of storing and showing answers.

Competitor pricing (current pages)

Tool	Plan	Price/mo	Prompts × engines × runs	Answers/mo	Price per answer
Otterly	Lite	$29	15 × 4 × daily	1,800	$0.016
Otterly	Standard	$189	100 × 4 × daily	12,000	$0.016
Otterly	Premium	$489	400 × 4 × daily	48,000	$0.010
Profound	Starter	$99	50 × 1 (ChatGPT) × daily	1,500	$0.066
Profound	Growth	$399	100 × 3 × daily	9,000	$0.044
Surfer	AI Search Analytics	$82	50 × 5 × daily	7,500	$0.011
Rankscale	Pro / Growth	$99 / $385	credit-based	up to 4,800 / 22,000	$0.021 / $0.018
AthenaHQ	Starter	$295	3,600 credits (1 credit = 1 answer)	3,600	$0.082
Scrunch	Starter / Growth	$300 / $500	350 / 700 × 7 platforms	frequency not stated	—

Peec doesn't show prices on its page; it bills by prompts × models × run frequency.

Our cost per answer (2 engines, paid rates)

- **Without web search:** ≈ **$0.0015** (gpt-5-nano ≈ $0.0007, Gemini Flash-Lite ≈ $0.002).
- **With web search (answers include sources):** ≈ **$0.011** on ChatGPT. The Gemini equivalent is legally blocked (above).

Suggested pricing

Plan	Price/mo	What's included	Answers/mo	Our cost, no search	Our cost, ChatGPT search
Starter	$79	50 prompts × 2 engines × weekly	~430	<$1	~$3
Growth	$299	100 × 2 × daily	6,000	~$9	~$40
Agency	$799	300 × 2 × daily, multiple brands	18,000	~$27	~$120

- Model API cost is small (under 15% of price); **hosting, support, sales and legal will be the bigger costs**.
- With web search on every answer, our cost per answer (≈ $0.011) matches or exceeds Otterly and Surfer's price per answer. Daily sourced tracking can't match their low prices, so offer weekly runs for sourced answers, or charge more for them.
- We cover 2 engines; competitors cover 4–9. Price below Profound and Athena, around Otterly Standard per engine, and stand out on transparency: fractions, published definitions and evidence behind every figure.

Our costs are estimates from token counts in stored answers and current list prices. Competitor prices come from their pricing pages today and may change.